

Resumo de Estudo — Aula 13

Algoritmo CYK (Cocke–Younger–Kasami) • Seção 7.2

Na aula anterior aprendemos a colocar uma gramática na Forma Normal de Chomsky. Agora vamos usar essa forma para resolver um problema central: dada uma gramática G na FNC e uma cadeia w , **decidir se w pertence a $L(G)$** — isto é, se G consegue gerar w . O algoritmo CYK faz isso de forma eficiente, em tempo polinomial.

1. O problema e a ideia geral

Entrada: uma gramática G na FNC (variável inicial S_0) e uma cadeia $w = w_1w_2\dots w_n$. **Saída:** aceitar se $w \in L(G)$, rejeitar caso contrário.

Vimos que testar todas as derivações de $2n-1$ passos resolve o problema, mas com custo **exponencial**. O CYK evita isso usando **programação dinâmica**: decompõe o problema em subproblemas menores (as subcadeias) e reaproveita as respostas.

Casos triviais (resolvidos direto pelas propriedades da FNC)

- Se $w = \varepsilon$: G gera ε **se e somente se** existir a regra $S_0 \rightarrow \varepsilon$.
- Se $w = a$ (um único terminal): G gera « a » sse existir a regra $S_0 \rightarrow a$. (Mais geral: uma variável V deriva o terminal « a » sse G tiver a regra $V \rightarrow a$.)

Os casos interessantes são cadeias de comprimento $n \geq 2$, tratados pela tabela abaixo.

2. A estrutura de dados: a matriz M

O coração do CYK é uma **matriz triangular superior** M . A célula $M(i, j)$ guarda a **lista de variáveis de G que derivam a subcadeia $w_iw_{i+1}\dots w_j$** (todos os pedaços da cadeia que vão da posição i até a posição j).

Por exemplo, $M(4,7)$ = lista das variáveis que derivam a subcadeia $w_4w_5w_6w_7$. As células abaixo da diagonal (onde $i > j$) não são usadas, porque não existe subcadeia "para trás".

Regra de aceitação

Depois de preencher toda a matriz, a resposta é imediata: se a variável inicial $S_0 \in M(1, n)$ — ou seja, se S_0 deriva a cadeia inteira, da posição 1 à n — então **aceite**; senão, **rejeite**.

O preenchimento é feito em **ordem crescente do tamanho das subcadeias**: primeiro todas as de tamanho 1, depois as de tamanho 2, tamanho 3, ..., até a cadeia completa (a célula $M(1, n)$). É a essência do bottom-up.

3. Como preencher a matriz

Subcadeias de tamanho 1 (a diagonal principal)

Para cada posição i de 1 a n : olhe o terminal $b = w_i$ e, para toda regra $V \rightarrow b$, adicione a variável V em $M(i, i)$. (Aqui usamos a propriedade da FNC: V deriva o terminal b sse existir a regra $V \rightarrow b$.)

Subcadeias de tamanho $L \geq 2$ (o passo indutivo)

Para preencher $M(i, j)$ de uma subcadeia de tamanho L , testamos **todos os pontos de quebra k** entre i e $j-1$. Cada k divide a subcadeia em duas partes: $w_i \dots w_k$ (já calculada em $M(i, k)$) e $w_{k+1} \dots w_j$ (já calculada em $M(k+1, j)$). Então, para toda regra $V \rightarrow WX$ com $W \in M(i, k)$ e $X \in M(k+1, j)$, adicionamos V em $M(i, j)$.

Por que isso funciona (Propriedade 2 da FNC)

Lembre da aula passada: se V deriva uma subcadeia, então existe um ponto de quebra que a divide em duas metades geradas por duas variáveis W e X , com $V \rightarrow WX$. O CYK simplesmente **testa todos os pontos de quebra possíveis** e todas as regras $V \rightarrow WX$, aproveitando as células já preenchidas para as subcadeias menores.

Esquematicamente, para preencher $M(3, 6)$ testamos as três quebras possíveis: em $k=3$ ($w_3 \mid w_4 w_5 w_6$), em $k=4$ ($w_3 w_4 \mid w_5 w_6$) e em $k=5$ ($w_3 w_4 w_5 \mid w_6$), combinando as variáveis das células correspondentes através das regras $V \rightarrow WX$.

4. O algoritmo consolidado

Reunindo tudo, este é o CYK completo:

```
Entrada: G na FNC (inicial  $S_0$ ), cadeia  $w = w_1 \dots w_n$ 

1. se  $w = \varepsilon$ : aceite sse existe  $S_0 \rightarrow \varepsilon$  ; senão rejeite
2. se  $w = a$  (um terminal): aceite sse existe  $S_0 \rightarrow a$ 
3. para  $i = 1$  até  $n$ :                                # subcadeias de tamanho 1
4.      $b = w_i$ 
5.     para toda regra  $V \rightarrow b$ : adicione  $V$  em  $M(i,i)$ 
6. para  $L = 2$  até  $n$ :                                # tamanho da subcadeia
7.     para  $i = 1$  até  $n-L+1$ :                          # posição inicial
8.          $j = i + L - 1$                              # posição final
9.         para  $k = i$  até  $j-1$ :                        # ponto de quebra
10.            para toda regra  $V \rightarrow WX$  com
                 $W \in M(i,k)$  e  $X \in M(k+1,j)$ :
                adicione  $V$  em  $M(i,j)$ 
11. se  $S_0 \in M(1,n)$ : aceite ; senão rejeite
```

5. Exemplo trabalhado passo a passo

Vamos rodar o CYK para a linguagem das cadeias com **a mesma quantidade de a's e b's**, testando a cadeia **w = aabab b** (isto é, a a b a b b, $n = 6$). A gramática original e sua FNC:

Original: $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

FNC utilizada

$S0 \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \epsilon$

$S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$

$T \rightarrow SB \qquad U \rightarrow SA$

$A \rightarrow a \qquad B \rightarrow b$

Passo 1 — diagonal (tamanho 1). Cada posição recebe a variável do seu terminal: onde há «a» entra A (regra $A \rightarrow a$); onde há «b» entra B (regra $B \rightarrow b$). Assim $M(i,i)$ fica: A, A, B, A, B, B.

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	∅	∅	∅	∅	∅
2		A	∅	∅	∅	∅
3			B	∅	∅	∅
4				A	∅	∅
5					B	∅
6						B

Matriz após o passo 1 (só a diagonal preenchida).

Passo 2 — tamanho 2. Para cada par adjacente, testamos as regras $V \rightarrow WX$. Exemplos: $M(2,3)$ recebe a subcadeia «ab» → como $M(2,2)=A$ e $M(3,3)=B$, e há a regra $S \rightarrow AB$ (e $S0 \rightarrow AB$), colocamos $\{S0, S\}$. Já $M(1,2) = \text{«aa»}$ → precisaríamos de uma regra $V \rightarrow AA$, que não existe, então fica ∅.

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
2		A	S0, S	$\emptyset$	$\emptyset$	$\emptyset$
3			B	S0, S	$\emptyset$	$\emptyset$
4				A	S0, S	$\emptyset$
5					B	$\emptyset$
6						B

Após o tamanho 2. As subcadeias «ab» (posições 2-3, 3-4) e «ba» (4-5) geram {S0,S} pelas regras $S \rightarrow AB$ e $S \rightarrow BA$. Os pares «aa» (1-2) e «bb» (5-6) ficam $\emptyset$, pois não há regra $V \rightarrow AA$ nem $V \rightarrow BB$.

Passos 3 a 5 — tamanhos 3, 4 e 5. Continuamos combinando, sempre testando todos os pontos de quebra. Vão surgindo as variáveis auxiliares U (regra $U \rightarrow SA$) e T (regra $T \rightarrow SB$) nas subcadeias maiores, e $\{S_0, S\}$ nas subcadeias balanceadas.

Passo final — tamanho 6 (a cadeia inteira). Preenchemos $M(1,6)$. Testando as quebras, encontramos uma combinação que gera S_0 . Como $S_0 \in M(1,6)$, a cadeia é **aceita**. A matriz completa fica:

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	S_0, S
2		A	S_0, S	U	S_0, S	T
3			B	S_0, S	T	$\emptyset$
4				A	S_0, S	T
5					B	$\emptyset$
6						B

Matriz M completa. A célula destacada $M(1,6)$ contém $S_0 \rightarrow$ a cadeia $aabab\ b$ é aceita. As células $\emptyset$ representam subcadeias que nenhuma variável deriva.

6. Comentários finais

Complexidade de tempo

Sejam n = comprimento da cadeia, r = número de regras, v = número de variáveis. O passo das subcadeias de tamanho 1 custa $O(r \cdot n)$. O bloco principal tem três laços aninhados (tamanho, i , k), cada um $O(n) \rightarrow O(n^3)$ iterações, e cada uma testa as regras a custo $O(r)$. Logo o total é **$O(r \cdot n^3)$** — ou simplesmente **$O(n^3)$** quando r é pequeno em relação a n . Muito melhor que o exponencial da força bruta.

- **Bônus 1:** a matriz M não diz só se a cadeia inteira pertence à linguagem — ela revela **todas as subcadeias** que pertencem a $L(G)$ (basta ver onde S_0 aparece). Útil para buscar padrões em cadeias de tamanho moderado.
- **Bônus 2:** esta versão é apenas um **reconhecedor** (aceita/rejeita); não constrói a árvore sintática. Para isso, cada variável inserida precisaria guardar ponteiros para as células de onde veio (as variáveis W e X que a geraram), e depois se faz uma busca a partir de $M(1,n)$.

Exercícios resolvidos

Exercício 1 — Rode o CYK para a cadeia «ab» com a FNC do exemplo

A cadeia «ab» tem $n = 2$. Passo 1 (diagonal): $M(1,1) = \{A\}$ (pois $w_1 = a$) e $M(2,2) = \{B\}$ (pois $w_2 = b$). Passo tamanho 2: $M(1,2)$ testa a única quebra $k=1$, com $W \in M(1,1) = \{A\}$ e $X \in M(2,2) = \{B\}$. Existe regra $V \rightarrow AB$? Sim: $S \rightarrow AB$ e $S0 \rightarrow AB$. Logo $M(1,2) = \{S0, S\}$.

Como $S0 \in M(1,2)$, a cadeia «ab» é **aceita** (ela tem um «a» e um «b» — quantidades iguais). ✓

Exercício 2 — Rode o CYK para «aab» (deve rejeitar)

$n = 3$. Diagonal: $M(1,1)=\{A\}$, $M(2,2)=\{A\}$, $M(3,3)=\{B\}$.

Tamanho 2: $M(1,2) = \langle aa \rangle \rightarrow$ precisa de $V \rightarrow AA$; não existe $\rightarrow \emptyset$. $M(2,3) = \langle ab \rangle \rightarrow V \rightarrow AB$ existe $\rightarrow \{S0, S\}$.

Tamanho 3: $M(1,3)$ testa $k=1$ ($A \mid M(2,3)=\{S0, S\}$) e $k=2$ ($M(1,2)=\emptyset \mid B$). Para $k=1$ precisaríamos de $V \rightarrow A \cdot S0$ ou $V \rightarrow A \cdot S$; não há regra assim. Para $k=2$ uma das partes é $\emptyset$. Logo $M(1,3) = \emptyset$.

Como $S0 \notin M(1,3)$, a cadeia «aab» é **rejeitada** — de fato ela tem dois a's e um b, quantidades diferentes. ✓

Exercício 3 — Qual a complexidade de decidir « $w \in L(G)$ » com o CYK?

$O(r \cdot n^3)$, com $n = |w|$ e r = número de regras da gramática na FNC; ou $O(n^3)$ considerando r constante. Isso vem dos três laços aninhados (tamanho, posição inicial e ponto de quebra), cada um variando $O(n)$ vezes, com o teste de regras custando $O(r)$ por célula. É polinomial — a grande vantagem sobre testar as $2n-1$ derivações por força bruta, que é exponencial. ✓